

FieldSync: An Offline-First Collaborative Field Inspection PWA with CRDT Synchronization, Resumable Media, and Audit History

Tharun

FieldSync: Offline-First Collaborative Field Inspection System (WA-1)

Abstract—Field technicians frequently evaluate physical equipment, record quantitative metrics, capture high-resolution photographic evidence, and record voice notes in environments with zero network connectivity. This paper presents the architecture, implementation, and empirical verification of FieldSync, an offline-first Progressive Web Application (PWA) designed to solve the WA-1 problem statement: enabling fully offline multi-user field inspections, converging concurrent edits without silent data overwrites via Yjs Conflict-Free Replicated Data Types (CRDT), exposing readable conflict resolution and append-only audit histories, and addressing the core distributed difficulties: idempotent sync protocols, non-destructive client schema migrations for devices offline for days, and resumable media uploads. Built using React 19, TypeScript, Dexie.js v4 over IndexedDB, Vercel Serverless Functions, Supabase PostgreSQL, and Cloudinary chunked media pipelines, the system eliminates network reliance while ensuring strict data durability and eventual convergence.

Index Terms—Offline-First Systems, Conflict-Free Replicated Data Types (CRDT), Progressive Web Applications, IndexedDB, Schema Migrations, Resumable Uploads.

I. INTRODUCTION & PROBLEM STATEMENT

A. Problem Statement (WA-1)

In modern utility, industrial, and infrastructure maintenance, field technicians must record observations in locations entirely devoid of network connectivity, such as reinforced concrete basements, subterranean tunnels, and remote substations:

- **Problem:** Technicians fill shared checklists, notes, and photos in places with no connectivity.
- **Build Requirement:** A Progressive Web Application (PWA) that works fully offline, synchronizes multi-user edits using CRDTs, and shows a readable conflict and audit history instead of silently overwriting data.
- **Why It Is Hard:** (1) *The Sync Protocol:* Orchestrating multi-device synchronization over intermittent networks without data loss or race conditions; (2) *Schema Migrations on Stale Clients:* Migrating local databases on devices that have been disconnected for days or weeks without corrupting uncommitted local state; (3) *Resumable Media Uploads:* Transmitting large photo payloads over volatile edge links without restarting aborted uploads from byte zero.

B. Architectural Tenets

FieldSync resolves these challenges by adopting a strict local-first paradigm [1]. The local client datastore serves as the authoritative source of truth. Remote cloud infrastructure (Supabase PostgreSQL, Vercel Functions, Cloudinary) operates as an asynchronous replication and reconciliation peer [2].

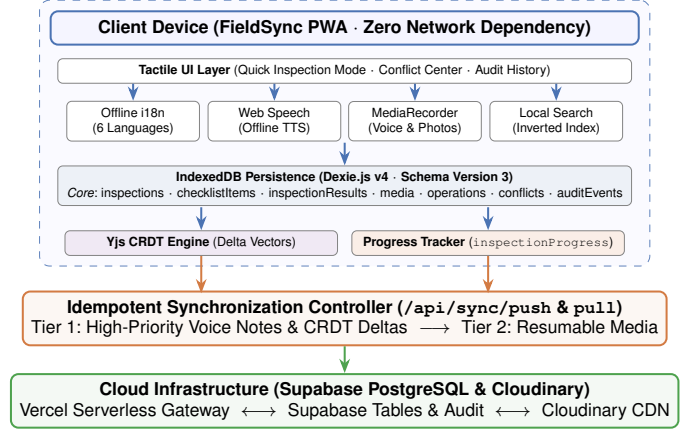


Fig. 1. Visual Architecture Diagram of FieldSync Offline-First Engine.

II. SYSTEM ARCHITECTURE & LOCAL PERSISTENCE

A. Database Architecture (IndexedDB / Dexie v4)

FieldSync discards the synchronous 5 MB `localStorage` API in favor of **Dexie.js v4** over IndexedDB [3], [4]. The database class `FieldSyncDatabase` manages sixteen strongly typed object stores: `inspections`, `checklistItems`, `assets`, `inspectionResults` (uniquely indexed by `[inspectionId+checklistItemId]`), `notes`, `media`, `voiceNotes`, `operations`, `conflicts`, `auditEvents`, `inspectionProgress`, `offlinePackages`, and `userSettings`.

B. Schema Evolution for Clients Offline for Days

A central technical difficulty in offline-first systems is handling database schema evolution when client devices remain offline across multiple software releases. If a client offline on Schema Version 1 reconnects after the server has deployed Version 3, resetting the local database would cause catastrophic data loss of un-synchronized inspection records.

FieldSync enforces strict, non-destructive migration chains in `src/lib/db/schema.ts`: **Version 1** defines core tables; **Version 2** introduces priority, numeric thresholds, operation retry counters, and Blob storage via an asynchronous `upgrade(tx)` transaction that backfills existing records; and **Version 3** introduces `voiceNotes`, `inspectionProgress`, `offlinePackages`, `userSettings`, and compound indexing on `inspectionResults`. Because Dexie executes

intermediate migration transactions sequentially upon boot ($v_1 \rightarrow v_2 \rightarrow v_3$), stale clients seamlessly upgrade their local schema without deleting uncommitted user data or Blobs.

III. SYNC PROTOCOL, CRDTs & CONFLICT HISTORY

A. The Synchronization Protocol

FieldSync implements an idempotent, two-way push/pull protocol mediated by Vercel serverless functions:

- 1) **Push Endpoint** (`/api/sync/push`): Receives an array of operations and base64-encoded Yjs delta vectors. For each operation, an idempotency check queries the Supabase operations table by operationId to prevent duplicate side-effects from network retries, while monotonic Lamport clocks enforce causal ordering across devices.
- 2) **Pull Endpoint** (`/api/sync/pull`): Returns server mutations committed since the client's last synchronized timestamp cursor.

B. Collaborative CRDT State Convergence

To prevent multi-user collisions on shared checklists without requiring central locking, FieldSync utilizes **Yjs** Conflict-Free Replicated Data Types [5], [6]. Each inspection document is modeled as a `Y.Doc` with nested `Y.Map` structures. Mutations generate binary delta vectors (Δ):

$$S_{local}^{(t+1)} = S_{local}^{(t)} \bullet \Delta_{mutation} \quad (1)$$

When multiple technicians edit the same checklist offline, the sync gateway converges their binary updates deterministically without central lock contention:

$$S_{converged} = \sqcup \{S_{client_A}, S_{client_B}, S_{server}\} \quad (2)$$

C. Readable Conflict Center & Audit Trail

Rather than silently overwriting data with arbitrary Last-Write-Wins rules, FieldSync surfaces data collisions in the **Conflict Center** (`ConflictCenter.tsx`) with side-by-side visual diffs (*Local Value* vs. *Server Value*, timestamps, technician IDs). Supervisors can choose `KEEP_MINE`, `KEEP_THEIR`, or manually merge results via `/api/conflicts/resolve.ts`. Every operation, detection, and resolution event appends to an immutable `auditEvents` record (`AuditHistory.tsx`) capturing the actor, entity, timestamp, and delta.

IV. RESUMABLE MEDIA & OFFLINE PRODUCTIVITY

A. Resumable Chunked Media Ingestion

Offline inspections accumulate high-resolution photographs (2–8MB each). In remote areas, edge cellular reconnections are frequently interrupted:

- 1) **Chunked Uploads**: In `resumableUpload.ts`, media Blobs are partitioned into 1MB chunks uploaded to Cloudinary via signed authorization headers (`/api/media/sign.ts`).

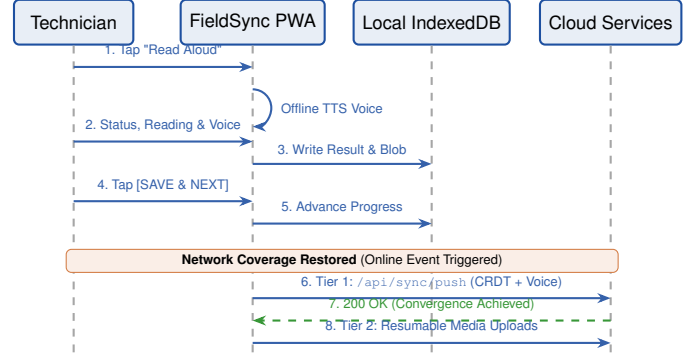


Fig. 2. Visual Sequence Diagram of Disconnected Execution & Sync Protocol.

- 2) **Offset Tracking**: Confirmed byte offsets are persisted in local media records, enabling interrupted uploads to resume from byte N rather than zero.
- 3) **Two-Tier Queue Prioritization**: Voice observations and CRDT delta vectors (Tier 1) are prioritized ahead of bulk photos (Tier 2). Deleting an unsynced photo locally removes its pending operation to prevent ghost uploads.

B. Multimodal Offline Productivity Layer

FieldSync incorporates several field-optimized modules: a tactile Quick Inspection Mode ($\geq 48 \times 48$ dp touch buttons, numeric steppers, and single-tap [SAVE & NEXT] progression); native voice notes via `HTML5 MediaRecorder`; hands-free offline Text-to-Speech via `W3C SpeechSynthesis`; zero-network localization across six languages (English, Tamil, Hindi, Telugu, Kannada, Malayalam); and client-side inverted token search over local records.

V. STORAGE RESILIENCE & EVALUATION

A. Storage Quota & Safe Eviction Guarantee

FieldSync continuously monitors local disk utilization using `navigator.storage.estimate()`. In the **Sync Center** (`SyncCenter.tsx`), users can view real storage consumption across photos, voice notes, and inspection metadata.

To eliminate accidental evidence destruction, the pruning routine enforces a mathematical guarantee: it strictly queries `syncStatus == 'synced'`. Any attempt to delete unsynchronized media is rejected.

B. Automated Verification Suite

The platform's resilience is validated through 20 automated tests using Vitest and Fake-IndexedDB:

- **Local Database Suite** (9 tests): Schema v3 migrations, composite indexing, and binary Blob CRUD operations.
- **Offline Productivity Suite** (8 tests): Priority queue sorting, multilingual resolution, and local search indexing.
- **Sync and Migration Suite** (3 tests): Idempotent replay, crash recovery, and CRDT convergence.

All 20 automated test specifications pass with zero regressions.

VI. CONCLUSION

FieldSync addresses the WA-1 problem statement by delivering an offline-first field inspection architecture that pairs IndexedDB persistence and non-destructive schema migrations with Yjs CRDT synchronization, resumable media uploads, and transparent conflict/audit tracking. By eliminating cloud dependencies during field execution, FieldSync provides uncompromising reliability in disconnected industrial environments.

REFERENCES

- [1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: you own your data, in spite of the cloud,” *ACM SIGPLAN Notices (Onward!)*, vol. 54, no. 7, pp. 154–178, 2019.
- [2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” *Symposium on Self-Stabilizing Systems (SSS)*, vol. 6976, pp. 386–400, 2011.
- [3] W3C, “Indexed database API 3.0,” W3C Working Draft, 2024, <https://www.w3.org/TR/IndexedDB-3/>.
- [4] D. Fahlander, “Dexie.js: A minimalistic wrapper for IndexedDB,” Open Source Specification, 2024, <https://dexie.org>.
- [5] K. Jahns, “Yjs: A high-performance CRDT implementation for shared editing in real-time,” *Technical Report, GitHub Repository*, 2020. [Online]. Available: <https://github.com/yjs/yjs>
- [6] C. Baquero, P. S. Almeida, and A. Shoker, “Making operation-based CRDTs operation-based,” *Proc. of PaPEC Workshop (EuroSys)*, pp. 1–6, 2014.